

The Seqlock

A Read-Optimized Snapshot Primitive, and Why Its Correctness Forces Fences Where Acquire/Release Cannot Reach

Robert (Bo) Hou · Robert's Technical Notes · June 2026

Abstract. The seqlock is the data structure of choice when a single writer must publish a frequently-updated, possibly-large snapshot — a market-data book, a configuration blob — to a swarm of latency-critical readers. This report does not present the final code as a *fait accompli*; it reconstructs the design as a sequence of tensions. We first show why the obvious tools (reader-writer locks, `std::atomic<BigStruct>`, MPMC queues) each fail, forcing the seqlock's central inversion: where a queue *prevents* data races, the seqlock *tolerates* race-like reads and *detects-and-retries*. We then confront the sharpest tension — that a naive seqlock is, by the letter of the C++ memory model, a data race and therefore undefined behavior, despite working flawlessly in practice — and resolve it. Finally we derive the memory ordering and discover that ordinary acquire/release is structurally insufficient here: because the protected payload is non-atomic, there is no atomic operation to bind the one-way barrier to, and correctness instead requires standalone *fences* that sandwich the payload access on both sides. The writer's release fence and the reader's acquire fence stand in exact duality.

1. The Problem, and Three Tools That Fail

The scenario is asymmetric in the extreme. One writer updates a snapshot — say the current top-of-book (bid, ask, sizes, timestamps), tens of bytes — at a furious rate. Dozens of strategy threads read it even more furiously. Reads vastly outnumber writes, and the read path must be as close to free as physically possible: no locks, no atomic writes, no blocking. Before building anything new, we should ask why the standard tools do not serve.

1.1 Reader-writer lock — readers block each other

A `shared_mutex` lets many readers hold the lock at once, which sounds right. But every reader still performs an atomic read-modify-write on the lock's state to acquire and release it, contending on the same cache line; readers serialize on that line and ping-pong it between cores. Worse, a writer must wait for all readers to drain, and readers must wait for the writer. On a hot read path traversed millions of times per second, paying a contended atomic on every read — and occasionally blocking — is exactly the cost we cannot afford.

1.2 `std::atomic<BigStruct>` — silently degrades to a lock

If the snapshot were small enough one could make it a single `std::atomic` and read it lock-free. But `std::atomic<T>` is only lock-free when `T` fits the platform's native atomic width — 8 bytes on x86, 16 with `cmpxchg16b`. A 64-byte book does not fit. The compiler then implements `atomic<BigStruct>` by hiding a lock inside, and `is_lock_free()` returns false. We are back to case 1.1, with readers silently taking a mutex. This failure is the seqlock's very reason for existing: **it gives a structure too large for a lock-free atomic the property of lock-free, non-blocking reads**, by moving atomicity off the payload and onto a tiny 8-byte counter.

1.3 MPMC queue — the wrong semantics entirely

A lock-free queue is the instinctive reach for “concurrent data structure,” but it answers a different question. A queue models a *stream*: every element must be consumed, nothing is dropped, and an element popped by one consumer is gone for the others. A market-data reader wants the opposite — the *latest* snapshot, with all stale values discarded. With a queue the writer would pile historical ticks the readers do not want; a reader wanting the newest would have to drain the backlog, and pop semantics mean two readers cannot both obtain the same latest value. The seqlock simply overwrites one shared snapshot in place; `N` readers read the same current value simultaneously, with no backlog and no consumption. Stream versus snapshot is the dividing line, and market data is squarely snapshot.

The verdict of Section 1. Reader-writer locks contend and block; `atomic<BigStruct>` degrades to a hidden lock; MPMC queues model streams, not snapshots. The gap — a large, non-atomic snapshot that nonetheless supports lock-free, non-blocking, multi-reader access — is exactly the seqlock's niche.

2. The Central Inversion: Tolerate and Detect, Not Prevent

The SPSC ring buffer (the subject of a companion report) achieves correctness by *prevention*: a happens-before edge guarantees a reader never touches a slot the writer is still writing. The seqlock cannot afford prevention — locking the payload is precisely what we ruled out — so it inverts the philosophy. It lets the reader proceed *optimistically*, possibly reading a torn, half-updated snapshot, and then *detects* after the fact whether the read was clean, *retrying* if not. Prevention is replaced by detection-and-retry.

The detector is a single monotonically increasing **sequence counter**, `seq_`, with one invariant: it is *odd* while a write is in progress and *even* when the data is at rest.

- **Writer:** increment `seq_` to odd (“I am writing”), update the payload, increment `seq_` to even (“done”).
- **Reader:** snapshot `seq_` as `s1`, read the payload, snapshot `seq_` again as `s2`. Accept the read only if `s1 == s2` and `s1` is even; otherwise discard and retry.

The two reader checks divide the labor cleanly, and it is worth stating their separate meanings because they are easy to conflate. `s1 even` means no write was in progress *when the read began* — if it were odd, the writer was mid-update and the read is doomed, so we retry without even trusting the payload. `s1 == s2` means no write *completed during* the read — if the writer ran a full cycle while we were reading, the counter advances by at least two and the equality fails. Both holding together means the entire payload read fell inside a single quiescent window between writes, so the snapshot is internally consistent.

A common confusion, dispelled. The seqlock payload is *never empty* — it always holds the current snapshot. A reader therefore never waits for data to “arrive,” the way a queue consumer waits on an empty queue. The retry loop is not waiting for data; it is waiting for a *clean* read. `s1 == s2` (and `even`) is the exit condition — the read was clean, take it and go. Inequality or oddness is the *continue* condition — the snapshot was disturbed mid-read, drop it and immediately read another (one is always available).

3. The Sharpest Tension: Correct in Practice, Undefined in the Standard

Here the design meets its most uncomfortable truth, and it is the question a serious interviewer will press. The reader reads the non-atomic `data_` while the writer may be writing the same `data_`, with no lock and no happens-before edge between them. By the very criterion established for the ring buffer, this is a data race: two threads, same non-atomic object, at least one write, unordered. And a data race in C++ is undefined behavior.

The uncomfortable fact. A naive seqlock whose reader does `result = data_;` on a plain non-atomic `data_` is, by the letter of C++11, a data race — hence UB — even though it works flawlessly on every real compiler and CPU. The retry does not save it: the read *action* already happened concurrently with the write; discarding its *result* afterward cannot un-commit the UB. The undefined behavior is incurred at the moment of the read, not at the moment of use.

There are three honest levels of answer, and a strong engineer should hold all three.

Level 1 — the practitioner. It works everywhere. Any torn or half-written bytes the reader observes are caught by the `seq_` comparison and discarded, never escaping into program logic; at the hardware level the payload’s loads and stores are ordinary memory accesses that do not “explode.” The Linux kernel has used seqlocks this way for decades, and HFT systems rely on them daily.

Level 2 — the standard-compliant repair. To make it defensibly correct, the payload must not be accessed as a plain non-atomic object during concurrency. The modern fix accesses `data_` through *relaxed atomic* operations — per-field atomics, or C++20 `atomic_ref`. Atomic accesses, even relaxed, never constitute a data race, so the UB vanishes; and because they are relaxed, they impose no ordering and cost nothing beyond a plain `mov`

on x86. The actual ordering is carried entirely by seq_.

Level 3 — the fences. Relaxed atomics on the payload remove the race but impose no ordering; ordering — the guarantee that the payload access stays *sandwiched* between the sequence reads — is supplied by fences, the subject of Section 4.

The inversion, restated as philosophy. SPSC *prevents* the race with a happens-before edge so the reader never touches a contended slot. The seqlock *tolerates* a race-like access and detects it after the fact with a version counter. One prevents; the other detects and retries. HFT uses both: prevent when prevention is cheap (SPSC), detect-and-retry when prevention is too expensive — a large snapshot read far more often than it is written (seqlock).

4. The Ordering: Why Acquire/Release Is Not Enough

It is tempting to assume the seqlock orders like the ring buffer — stores get release, loads get acquire, symmetric pairs. That assumption breaks in two places, and tracing why is the heart of the design.

4.1 The writer side

```
void store(const T& desired) {           // single writer
    uint64_t s = seq_.load(relaxed);
    seq_.store(s + 1, /* [1] */);         // enter write (odd)
    /* fence? */
    data_ = desired;                     // write payload
    seq_.store(s + 2, /* [2] */);         // done (even)
}
```

[2] is a release store, and that one is easy. Its job is to keep the preceding payload write from sinking below it, so that a reader observing the even seq_ is guaranteed the payload is fully written. Release — which welds *prior* writes above the store — does exactly this. No fence needed.

[1] is the trap. One instinctively writes seq_.store(s+1, release) for symmetry, but that release is *vacuous*: there is nothing before [1] to publish — the payload is written *after* [1], not before. What [1] actually needs is the *opposite* direction: it must stop the later payload write from floating *up* above the “I am writing” announcement, so a reader cannot see an even (quiescent) counter while the payload is already being mutated. Release does not constrain that direction — release welds prior writes down, it does not stop subsequent writes from rising. The correct tool is a standalone **release fence** placed after [1] and before the payload write:

```
seq_.store(s + 1, relaxed);               // [1] announce (relaxed is fine)
std::atomic_thread_fence(memory_order_release); // block payload write from rising
data_ = desired;
seq_.store(s + 2, release);               // [2] welds payload write down
```

4.2 The reader side — the dual

```
T load() const {           // multiple readers
    T result; uint64_t s1, s2;
    do {
        s1 = seq_.load(/* [3] */);
        result = data_;           // read payload
        /* fence? */
        s2 = seq_.load(/* [4] */);
    } while (s1 != s2 || (s1 & 1));
    return result;
}
```

The reader must keep result = data_ *sandwiched* between the two sequence reads — if the payload read drifts above the first or below the second, then s1 and s2 no longer bracket the true moment of the read, and the version check becomes meaningless.

[3] is an acquire load. After reading `s1`, the payload read must not be hoisted *above* it. Acquire — which stops *later* reads from rising above the load — does exactly this.

[4] is an acquire fence, not an acquire load. Here is the dual of the writer’s trap. The constraint at [4] is that the payload read must not sink *below* the second sequence read — and an acquire load constrains the wrong direction (acquire stops things rising, not sinking). The correct tool is a standalone **acquire fence** placed after the payload read and before the second sequence load; `s2` itself can then be a relaxed load, since the fence already anchors the ordering:

```
s1 = seq_.load(acquire);           // [3] block payload read from rising
result = data_;
std::atomic_thread_fence(memory_order_acquire); // block payload read from sinking
s2 = seq_.load(relaxed);           // value only; fence did the ordering
```

4.3 The deeper reason: nothing to bind the barrier to

The recurring surprise — a release fence on the writer, an acquire fence on the reader, where one expected plain release/acquire — has a single root cause. An acquire or release is a *one-way barrier bound to a specific atomic operation*; it orders that operation against the accesses before or after it. A fence is a *free-standing two-sided wall between two statements*, constraining every access that would cross it. The seqlock must order the `data_` access, but `data_` is *not* an atomic operation — it is a plain (or relaxed) access to a large non-atomic payload. There is no atomic operation on the payload to which an acquire or release could attach. With nothing to bind to, the only instrument that can fence the payload is a standalone fence, erected on both sides to sandwich it.

The portable rule. To order an access that is *itself atomic* (the ring buffer’s tail/head loads and stores), bind acquire/release directly to it — one-way is enough. To order an access that is *not atomic* (the seqlock’s payload, or any “small atomic flag guarding a large non-atomic blob”), there is nothing to bind to, so use a standalone fence to wall it off on the side that matters. Writer and reader fences stand in duality: the writer’s release fence keeps the payload write from rising above “writing,” the reader’s acquire fence keeps the payload read from sinking below the second check. This pattern — one atomic flag, one large non-atomic payload — recurs in RCU, double-buffering, and any large-object publication; the seqlock is its canonical instance.

4.4 Platform note

On x86-TSO the naive symmetric guess (release on both writer stores, acquire on both reader loads, no explicit fences) *happens to run correctly*, because the hardware already forbids store-store and load-load reordering — the very directions the fences guard. The fences are therefore nearly free on x86 (often compiling to nothing) yet essential on weakly-ordered architectures such as ARM, where the payload access really can drift out of the sandwich. Writing the fences explicitly is what makes the seqlock correct everywhere, not merely on the machine one happens to test on.

5. Synthesis

The seqlock earns its place by refusing every conventional tool and inverting the usual contract. Where a lock would block readers and an `atomic<BigStruct>` would secretly become a lock, the seqlock keeps the read path lock-free and non-blocking by moving atomicity onto an 8-byte counter and letting the large payload stay non-atomic. Where a queue would model a stream, the seqlock models a snapshot. Where the ring buffer prevents races, the seqlock tolerates and detects them. And where acquire/release would suffice for an atomic flag, the non-atomic payload has nothing to bind a one-way barrier to, so correctness falls to a dual pair of fences that sandwich the payload — a release fence on the writer, an acquire fence on the reader. The structure is small; the reasoning that makes it correct spans the entire distance from cache contention to the C++ abstract machine, and that is precisely why it is worth deriving rather than merely copying.